FICHE DE RÉVISION

Introduction to Business Intelligence

Chapitre 4 : From ETL to OLAP – Building Reliable Data Warehouses

Lab 5 : Data Integration avec Talend Open Studio

Dorline TIAKO OKASSIE

ING4 – Data & Intelligence Artificielle – Groupe 01

ECE Paris – Année 2026

Comment utiliser cette fiche

Partie 1 – Les notions du cours à connaître (ETL, qualité des données, OLAP).

Partie 2 – Comment chaque notion a été appliquée dans le Lab 5 avec Talend.

Partie 3 – Les résultats obtenus et leur interprétation.

Partie 4 – Les pièges rencontrés, un glossaire Talend et des questions d'entraînement corrigées.

0. Objectifs

0.1 Objectifs du cours (Chapitre 4)

D'après le support de cours, à la fin du chapitre vous devez être capable de :

1. Comprendre le **rôle des processus ETL** dans l'alimentation de l'entrepôt de données (Data Warehouse).
2. Savoir garantir la **qualité des données** par le nettoyage, la transformation et la **gouvernance**.
3. Connaître les méthodes pour traiter les données **manquantes, bruitées, incohérentes et dupliquées**.
4. Découvrir les **opérations OLAP** pour l'analyse multidimensionnelle (restructuration, granularité, slice, dice).
5. Relier **ETL, gouvernance et OLAP** pour une prise de décision fiable.

0.2 Objectifs du Lab 5 (Data Integration)

Exercice	Étape ETL	Objectif
Exercice 1	**Extract**	Lire un fichier CSV local (movies.csv) et afficher les données dans la console Talend.
Exercice 2	**Transform**	Joindre les films et les réalisateurs pour ne garder que les films valides (avec réalisateur).
Exercice 3	**Load**	Séparer les films valides et invalides et les charger dans deux fichiers de sortie (CSV et XML).

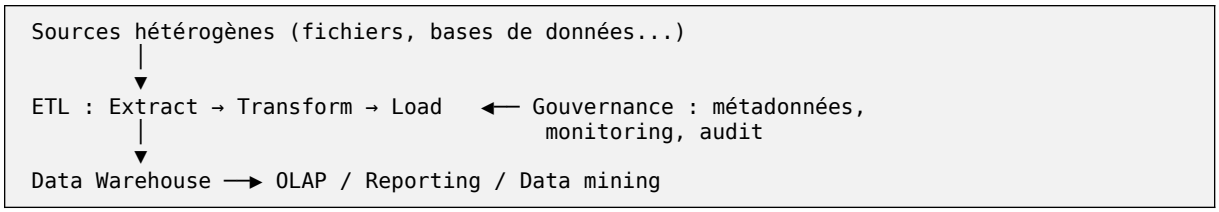
> **À retenir**
>
> Le Lab 5 met en pratique la **partie ETL** du chapitre 4 (extraction, transformation/nettoyage, chargement).
>
> La **partie OLAP** du cours n'est pas manipulée dans ce lab : elle intervient **après** l'ETL, une fois les données propres chargées dans l'entrepôt.

Partie 1 – Les notions du cours

1.1 Architecture d'un Data Warehouse

L'architecture multi-niveaux d'un entrepôt de données met l'**ETL au centre** : il **extrait** les données de sources hétérogènes, les **transforme** pour assurer cohérence et qualité, puis les **charge** dans l'entrepôt.

La **gouvernance des données** se traduit par les **métadonnées**, la **surveillance (monitoring)** et l'**audit**, qui garantissent la fiabilité, la traçabilité et la conformité des données. Grâce à l'ETL et à la gouvernance, les données peuvent ensuite être exploitées par les outils d'analyse, de reporting et de data mining.



1.2 L'alimentation du Data Warehouse (ETL)

Une fois le modèle de données conçu, il faut **implémenter** l'entrepôt en collectant des données de **sources multiples et souvent hétérogènes**. Ce processus est guidé par des **règles stockées sous forme de métadonnées**, essentielles au fonctionnement, à la maintenance et à l'administration de l'entrepôt.

Un **outil ETL** automatise l'alimentation et le chargement de l'entrepôt. **ETL = Extract, Transform, Load** :

Étape	Ce qu'elle fait	Objectif
Extraction	Récupérer les données des différentes sources	Accéder à des sources multiples et variées
Nettoyage & Transformation	Assurer la qualité et la cohérence des données	Traiter des formats variés, détecter et corriger les incohérences
Chargement	Transférer les données traitées dans l'entrepôt	Charger les données préparées dans le DW

1.3 Extraction

- **Identifier les sources** est la première étape : il faut décider quelles données de production inclure. Toutes les données sources ne sont pas utiles à la décision.
- Exemple du cours : au lieu de stocker l'adresse complète, il peut être plus pratique d'extraire seulement le **code postal**.
- Un **extracteur** (script, logiciel ou application) est mis en place pour **chaque source**. Il sélectionne et récupère les données, les **convertit dans un format commun** et utilise des interfaces de connexion comme **ODBC** ou **JDBC** (des pilotes qui servent de traducteur entre une application et une base de données).

Type	Définition	Cas d'usage
Extraction complète	Capture **toutes** les données à un instant donné.	Chargement initial de l'entrepôt ; rafraîchissement complet quand la source a beaucoup changé.

Type	Définition	Cas d'usage
Extraction incrémentale	Capture **seulement** les données ajoutées ou modifiées depuis le dernier chargement.	**Temps réel** (mises à jour immédiates) ou **différée/périodique** (mises à jour planifiées).

1.4 Transformation et nettoyage des données

> **Point clé du cours**
>
> Garantir la **cohérence et l'intégrité** des données est une étape critique.
>
> La justesse des analyses dépend directement de la **qualité des données** : des données de mauvaise qualité donnent des analyses fausses.
>
> Les études montrent que **5 % à 30 %** des données sources contiennent des erreurs.

Les **4 grands problèmes** des données réelles :

- **Incomplètes** : valeurs manquantes ou nulles.
- **Bruitées** : erreurs, exceptions, fausses saisies, fautes de frappe.
- **Incohérentes** : valeurs et descriptions qui ne correspondent pas, formats incohérents.
- **Dupliquées** : enregistrements redondants ou répétés.

Tableau de synthèse des méthodes (à connaître par cœur)

Problème	Définition	Techniques / Exemples
Données manquantes	Certains attributs n'ont pas de valeur.	Ignorer l'enregistrement, remplir à la main, remplacer par une constante (« unknown »), utiliser la moyenne ou la valeur la plus probable.
Données bruitées	Erreurs ou variations aléatoires dans les mesures.	Lissage, **binning** (moyenne/médiane), **clustering** (suppression des valeurs aberrantes), **régression**.
Conflits	Incohérences entre sources.	Conflit **structurel** (âge vs date de naissance), conflit de **représentation** (YYYY-MM-DD vs MM/DD/YY).
Conversion	Formats différents pour le même attribut.	Dates : « 24 FEB 2011 » vs « 24/02/2011 » vs « 02/24/2011 ».
Décodage des champs	Même donnée avec des noms ou codes différents.	Noms : num-client vs client-id ; valeurs : M/F vs male/female vs 1/2 ; codes : AC, IN, SU = actif, inactif, suspendu.
Données dupliquées	Champs redondants ou chevauchants entre sources.	**Découper** un champ complexe (nom complet → prénom, 2e prénom, nom) ; **fusionner** des sources (source 1 : code + type, source 2 : taille + couleur).

1.5 Les opérations OLAP

Une fois les données chargées dans l'entrepôt, on les analyse sous forme de **cube multidimensionnel**. Le cours classe les opérations OLAP en **3 familles** :

Type d'opération	Description	Exemples	Objectif
Restructuration	Change la représentation des données pour une nouvelle perspective.	Rotate/Pivot, Switch, Split, Nest, Push	Voir les données sous différents angles.

Type d'opération	Description	Exemples	Objectif
Granularité	Ajuste le niveau de détail.	Roll-up, Drill-down	Résumer ou détailler les données.
Ensemblistes	Extrait, sélectionne ou manipule des sous-ensembles.	Slice, Dice, Join (Drill-across)	Combiner ou filtrer pour une analyse précise.

a) Restructuration

- **Rotate / Pivot** : fait tourner le cube autour d'un axe pour afficher d'autres faces. Exemple : avant, l'Année est la dimension principale ; après, le Produit devient la dimension principale, ce qui permet de comparer les ventes par produit à travers les années et les régions. C'est une sélection de **faces**, pas de membres.
- **Switch (permutation)** : échange la **position des membres** d'une dimension. Les données ne changent pas, seul l'ordre change.
- **Split (division)** : affiche chaque tranche du cube séparément ; le cube 3D devient un **ensemble de tableaux 2D** (ex. un Split sur Région donne 4 tableaux).
- **Nest (imbrication)** : regroupe tous les membres et mesures dans une **seule vue 2D**, quel que soit le nombre de dimensions.
- **Push** : combine des **membres de dimension avec les mesures** ; les membres deviennent du contenu de cellule.

b) Granularité

La granularité est la structuration hiérarchique des données en **niveaux de détail**. Le niveau le plus bas correspond aux données brutes de l'entrepôt.

- **Roll-up (agrégation)** : monte d'un niveau dans la hiérarchie, donc **moins de détail**. Ex. : ventes journalières → totaux mensuels ou annuels.
- **Drill-down (forage)** : l'inverse du roll-up, donc **plus de détail**, en affinant une dimension ou en en ajoutant une. Ex. : ventes annuelles → trimestrielles ; régions → villes ; analyser week-ends vs jours de semaine.

c) Opérations ensemblistes

- **Slice** : projection selon **une** dimension ; on **fixe une dimension à une seule valeur** et on obtient une tranche 2D. Ex. : uniquement l'année 2023.
- **Dice** : sélectionne un **sous-cube** en filtrant **plusieurs dimensions**. Ex. : ventes 2022–2023, en Europe, catégorie Electronics.

d) Exemple du cours : le cube des ventes

Dimensions : **Ville** (Köln, Berlin, Lyon, Paris), **Temps** en trimestres (Q1 = Jan+Feb+Mar, Q2, Q3), **Produit** (Beverages, Produce, Condiments, Seafood). Mesure : **quantité vendue**.

```
-- Une face du cube = une requête d'agrégation
SELECT SUM(Quant) FROM Sales WHERE Date = Q1 GROUP BY P_Nm;
   → Köln, Q1 : Beverages 24 | Produce 18 | Condiments 28 | Seafood 14

-- DICE : sous-cube sur 2 villes et 2 trimestres
DICE(Sales, (City = 'Lyon' OR City = 'Berlin')
        AND (Time.Quarter = 'Q1' OR Time.Quarter = 'Q2'))
   → ventes des 2 premiers trimestres pour Lyon et Berlin
```

Conclusion du chapitre (cours)

Le chapitre couvre les processus **ETL** et leur rôle dans l'intégration de données hétérogènes, l'importance de la **gouvernance** (qualité, cohérence, intégrité), les techniques de traitement des données **manquantes, bruitées et incohérentes**, et les **opérations OLAP** (restructuration, granularité, slice, dice). Ensemble, ces méthodes assurent des analyses fiables pour la décision.

Partie 2 – Application dans le Lab 5

2.1 Le contexte et les données

Fichier	Format	Contenu	Particularités
movies.csv	Séparateur `;`, avec en-tête, fin de ligne UNIX	1 455 films : movieID, title, releaseYear, url, directorID	directorID vide pour 1 200 films
Directors.txt	Séparateur `,`, sans en-tête, fin de ligne WINDOWS	204 réalisateurs : id, nom	Espaces/tabulations parasites, accents perdus, doublon

Les deux sources sont **hétérogènes** (séparateurs, en-tête, fins de ligne différents) : c'est exactement la situation décrite dans le cours pour justifier l'ETL.

2.2 Tableau de correspondance Cours ↔ Lab

Notion du cours	Application concrète dans le Lab 5
Métadonnées (règles stockées, gouvernance)	Création des métadonnées « movies » et « directors » dans le Référentiel Talend : séparateur, en-tête, schéma et types sont définis une fois et réutilisés dans les 3 jobs.
Extracteur par source / format commun	Un composant **tFileInputDelimited** par fichier ; Talend convertit chaque ligne en colonnes typées (schéma).
Extraction complète	À chaque exécution, tout le fichier est relu (1 455 films, 204 réalisateurs) : pas d'incrémental ici.
Conflit de représentation / format	Séparateurs `;` vs `,`, en-tête vs pas d'en-tête, UNIX vs WINDOWS → réglés dans chaque métadonnée.
Décodage des champs (num-client vs client-id)	`directorID` (movies) et `director_id` (directors) désignent la même donnée : on les relie par la jointure du tMap, avec le même type String.
Données manquantes (ignorer l'enregistrement)	Les 1 200 films sans directorID sont **écartés** par l'**Inner Join** (Exercice 2), puis **capturés** dans un fichier de rejets (Exercice 3).
Données bruitées (erreurs de saisie)	Espaces et tabulations autour des noms → nettoyés avec `row2.director_name.trim()` dans le tMap.
Données dupliquées	Titres en double (ex. Butcher Boy ×2) et Barbet Schroeder sous 2 IDs → identifiés (non supprimés dans le lab).
Chargement dans la cible	**tFileOutputDelimited** (valid_movies.csv) et **tFileOutputXML** (invalid_movies.xml).
Gouvernance / audit	Les rejets ne sont pas perdus : ils sont stockés pour être analysés ; les compteurs de lignes permettent de contrôler le traitement.

2.3 Exercice 1 – EXTRACT : lire et afficher

Objectif : lire `movies.csv` et afficher les données dans la console (étape Extract).

```
movies (tFileInputDelimited) ──row1 (Main)──▶ tLogRow_1 (console)
```

Étapes clés

1. Créer le projet **PW1**.
2. Créer la métadonnée **movies** (Fichier délimité) : séparateur `;`, cocher « Définir la ligne d'en-tête comme nom des colonnes », schéma à 5 colonnes (directorID en String).
3. Créer le job **Exercise1** : glisser la métadonnée → **tFileInputDelimited** ; ajouter un **tLogRow** ; relier par **Ligne → Principal (Main)**.
4. Exécuter (**F6** ou ▶ Exécuter) et documenter.

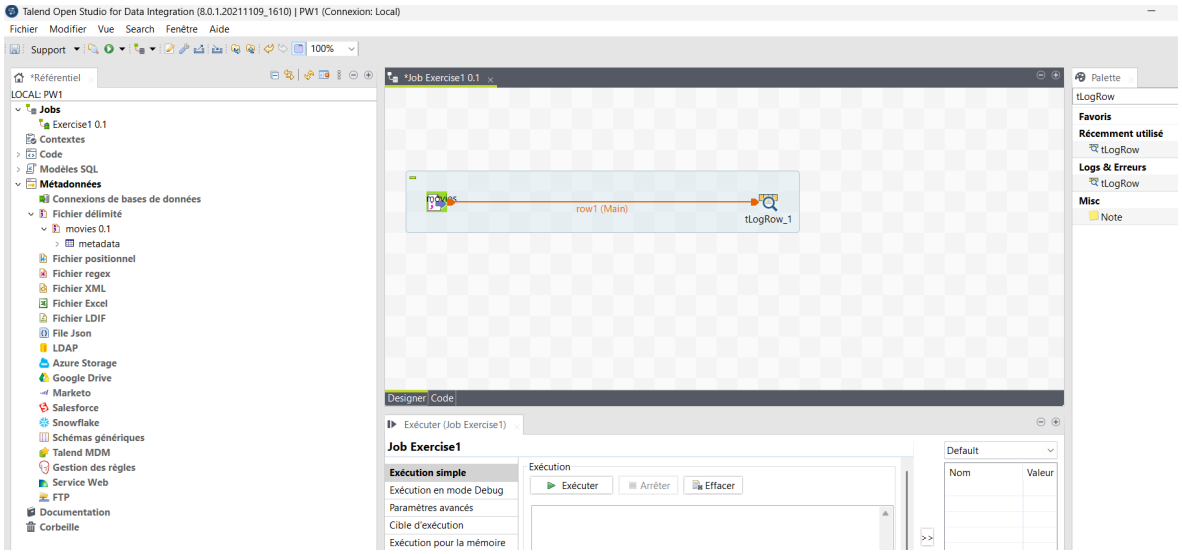


Figure 1 : *Job Exercise1 : tFileInputDelimited → tLogRow*

> **Lien avec le cours**
>
> Cet exercice illustre l'**extraction** : un extracteur (tFileInputDelimited) accède à une source et convertit ses données dans un format commun (le schéma Talend) grâce aux **métadonnées**.

2.4 Exercice 2 – TRANSFORM : jointure et filtrage

Objectif : ne garder que les films ayant un réalisateur valide (étape Transform).

```
movies    ──row1 (Main)──▶ ┐
                           ├─ tMap_1 ──output (Main)──▶ tLogRow_1
directors ──row2 (Lookup)─▶ ┘   Inner Join : row1.directorID = row2.director_id
```

Étapes clés

1. Créer la métadonnée **directors** : séparateur `,`, **sans** en-tête, format WINDOWS, encodage UTF-8 ; renommer Column0/Column1 en **director_id / director_name** ; director_id en **String**.
2. Job **Exercise2** : movies et directors → **un seul tMap** ; le 1er lien devient **Main**, le 2e devient automatiquement **Lookup** (pointillés) ; tMap → tLogRow avec une sortie nommée **output**.
3. Dans le tMap : glisser `directorID` sur la **Clé d'expr.** de `director_id` (jointure) ; clé à molette de row2 → **Join Model = Inner Join**.
4. Glisser **title** et **director_name** vers output ; ajouter `.trim()` pour nettoyer les noms.
5. Appliquer → OK → répondre **Yes** à « propager les modifications » → exécuter.

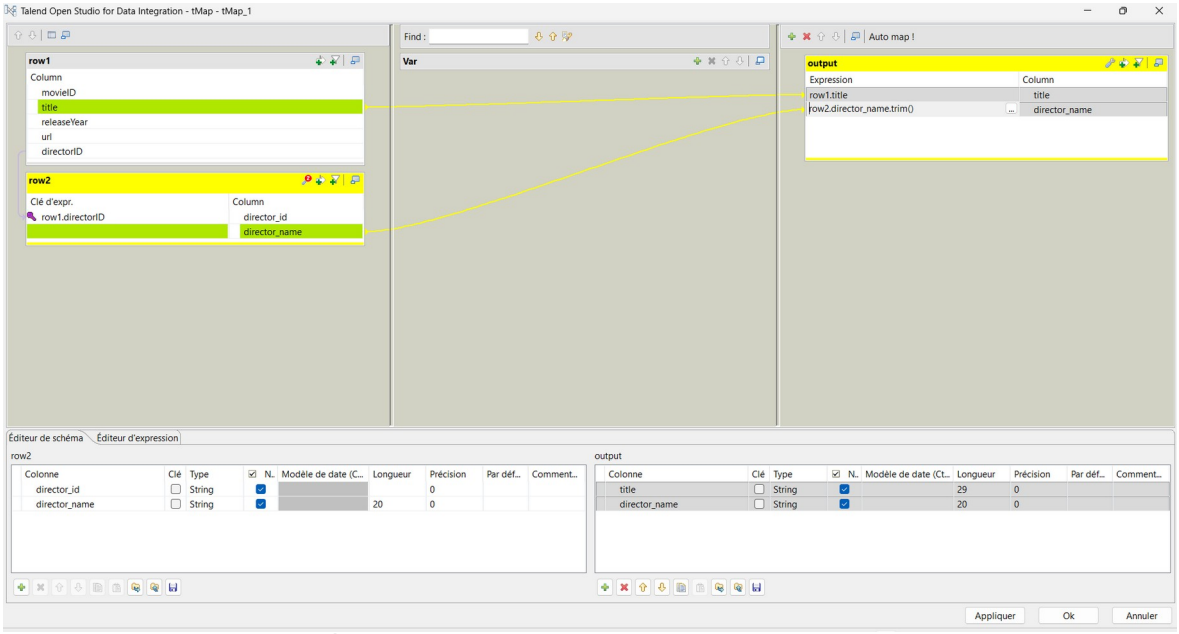


Figure 2 : Éditeur tMap de l'exercice 2 : jointure sur directorID et Inner Join

Inner Join vs Left Outer Join (question classique)

Join Model	Comportement	Résultat dans le lab
Left Outer Join (par défaut)	Garde **toutes** les lignes du flux principal ; si pas de correspondance, les colonnes du lookup sont **null**.	1 455 lignes, dont 1 200 avec director_name vide
Inner Join	Garde **uniquement** les lignes qui trouvent une correspondance dans le lookup.	**255 lignes** (films valides)

> **Lien avec le cours**
>
> Cet exercice illustre la **transformation** : intégration de **deux sources hétérogènes**, résolution d'un problème de **décodage des champs** (directorID ↔ director_id), traitement des **données manquantes** (enregistrements ignorés) et des **données bruitées** (`trim()`).

2.5 Exercice 3 – LOAD : valides et rejetés dans des fichiers

Objectif : charger les films valides et les films invalides dans deux fichiers de sortie (étape Load).

```
movies    ──row1 (Main)──►┐                ┌─ output_CSV ─► tFileOutputDelimited
                          ├─ tMap_1 ─┤                (valid_movies.csv)
directors ──row2 (Lookup)─►┘                └─ output_XML ─► tFileOutputXML
                                               (invalid_movies.xml)
```

Étapes clés

1. Job **Exercise3** : movies + directors → tMap → **2 nouvelles sorties** : **output_CSV** (vers tFileOutputDelimited) et **output_XML** (vers tFileOutputXML).
2. tMap : même jointure, **Inner Join**.
3. **output_CSV** : title + director_name, option **Catch lookup inner join reject = false** → reçoit les films **qui correspondent**.
4. **output_XML** : title, option **Catch lookup inner join reject = true** → reçoit les films **rejetés** par la jointure.

5. Nommer les fichiers en tapant le chemin entre guillemets :
   `"../TalendLab5/valid_movies.csv"` (cocher « Inclure l'en-tête ») et
   `"../TalendLab5/invalid_movies.xml"`.
6. Exécuter et vérifier les fichiers créés.

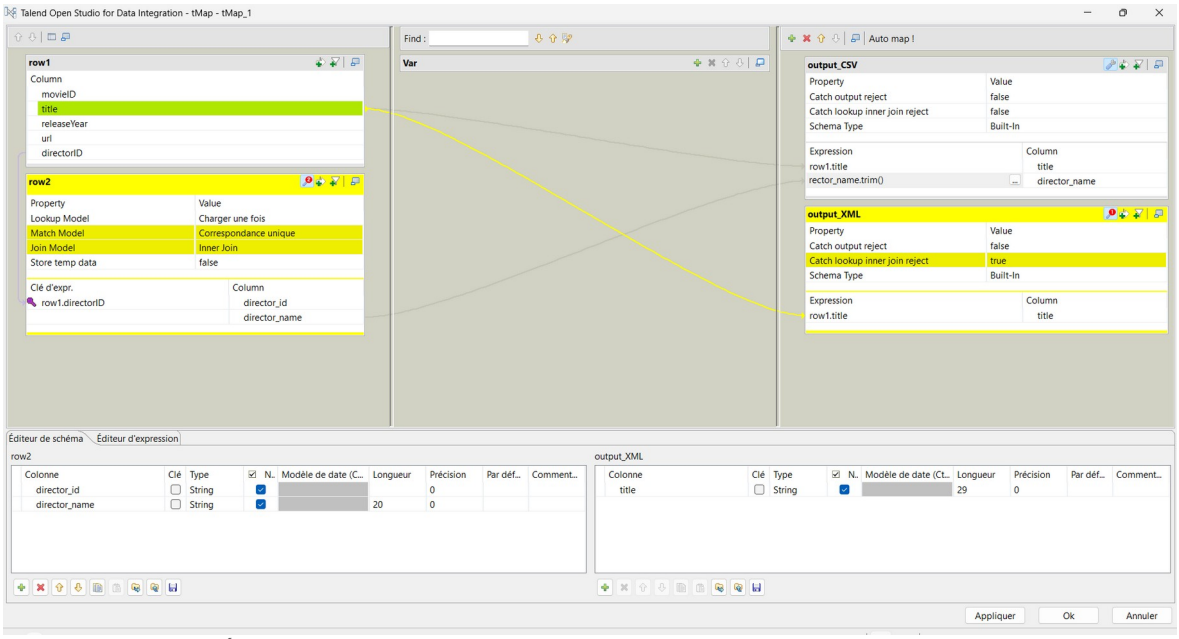


Figure 3 : *Éditeur tMap de l'exercice 3 : output_CSV (reject = false) et output_XML (reject = true)*

À retenir : Catch lookup inner join reject

false (par défaut) → la sortie reçoit les lignes qui **ont** trouvé une correspondance.

true → la sortie reçoit **uniquement les lignes rejetées** par l'Inner Join (sans correspondance).

Une seule jointure permet donc de **router** les données vers deux destinations différentes.

Lien avec le cours

Cet exercice illustre le **chargement** et la **gouvernance** : au lieu de perdre silencieusement les enregistrements incomplets, on les **conserve dans un fichier de rejets** pour pouvoir les **auditer** et les corriger à la source.

Partie 3 – Résultats finaux

3.1 Chiffres obtenus

Flux	Exercice 1	Exercice 2	Exercice 3
Films lus (row1)	**1 455**	**1 455**	**1 455**
Réalisateurs lus (row2)	–	**204**	**204**
Films valides (avec réalisateur)	–	**255** (console)	**255** → valid_movies.csv
Films rejetés (sans réalisateur)	–	éliminés	**1 200** → invalid_movies.xml
Code de sortie	0	0	0

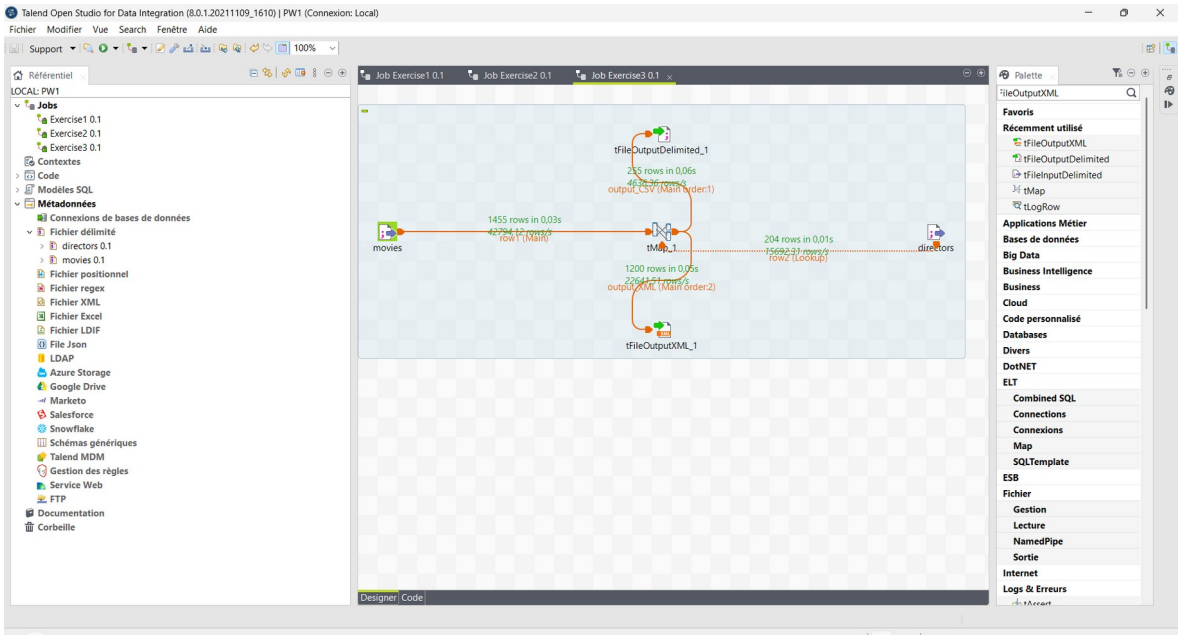


Figure 4 : *Exécution de l'exercice 3 : 1455 films → 255 valides (CSV) + 1200 rejetés (XML)*

3.2 Interprétation

- **255 + 1 200 = 1 455** : chaque film se retrouve dans **exactement un** des deux fichiers. Aucun enregistrement n'est perdu ni dupliqué par le traitement : c'est un **contrôle de cohérence** essentiel.
- Seuls **17,5 %** des films (255/1 455) ont un réalisateur : la source présente un **taux de données manquantes de 82,5 %** sur directorID, bien au-delà des 5–30 % cités dans le cours.
- Les fichiers produits sont corrects : `valid_movies.csv` (en-tête `title;director_name` + 255 lignes, noms nettoyés) et `invalid_movies.xml` (balises `<root>/<row>/<title>`, 1 200 films, caractères spéciaux encodés en `'` et `&`).

3.3 Problèmes de qualité observés (lien avec le chapitre 4)

Problème du cours	Observation dans les données	Traitement
Manquantes	1 200 films avec directorID vide	Rejetés par l'Inner Join, conservés dans le XML

Problème du cours	Observation dans les données	Traitement
Bruitées	Espaces/tabulations : « Gregg Araki », « Alex Sichel »	`.trim()` dans le tMap
Dupliquées	Titres répétés (Butcher Boy, Hugo Pool, Money Talks…) ; Barbet Schroeder IDs 13 et 14	Identifiés ; suppression possible avec tUniqRow
Conversion / encodage	Accents perdus dès la source : Cuar�n, Hallstr�m, C?r?monie, Mis?rables	Non récupérable dans Talend : corriger la source
Conflits de format	`;` vs `,`, en-tête vs non, UNIX vs WINDOWS	Deux métadonnées distinctes

3.4 Conclusion

Le Lab 5 a permis de réaliser un processus **ETL complet** avec Talend Open Studio : **Extract** (lecture d'un fichier délimité via une métadonnée réutilisable), **Transform** (jointure de deux sources hétérogènes dans un tMap avec Inner Join et nettoyage des valeurs) et **Load** (écriture des films valides dans un CSV et des rejets dans un XML). Ces données propres pourraient ensuite alimenter un entrepôt et être analysées avec les **opérations OLAP** du cours (par ex. un roll-up du nombre de films par réalisateur ou par année de sortie).

Partie 4 – Pièges, glossaire et entraînement

4.1 Pièges rencontrés et solutions

Problème	Cause	Solution
Plusieurs tMap créés par erreur (tMap_1, _2, _3)	Lien relâché sur une zone vide → Talend propose de créer un composant	Appuyer sur **Échap** ; viser exactement l'icône du composant cible
Double-clic sur le tMap sans effet	Composants superposés / tMap sans sortie	Supprimer les doublons, garder un seul tMap
Impossible de renommer les colonnes	On était à l'étape 3 (aperçu en lecture seule)	Renommer à l'**étape 4 – Schéma**
Directors.txt invisible avec Parcourir	Filtre réglé sur *.csv	Changer le filtre en *.* ou taper le nom
Une seule colonne dans l'aperçu	Séparateur resté sur Semicolon	Choisir **Comma**
Fenêtre « modules tiers requis »	Librairie jboss-marshalling-river manquante (lookup du tMap)	**Télécharger et installer**, accepter la licence Apache-2.0
« Fichier introuvable » avec [...] en sortie	La fenêtre Ouvrir n'accepte que des fichiers existants	Taper le chemin entre guillemets dans « Nom de fichier »
Erreur « file already exist », code 1	Option « Générer une erreur si le fichier existe déjà »	La décocher dans **Paramètres avancés**
Plus d'espace de travail visible	Double-clic sur l'onglet Référentiel (agrandi)	Re-double-cliquer sur l'onglet

4.2 Glossaire Talend

Terme	Signification
Projet / Référentiel	Espace qui contient les jobs, métadonnées, code... (ici le projet PW1).
Métadonnée (Fichier délimité)	Description réutilisable d'une source : chemin, séparateur, en-tête, encodage, schéma.
Schéma	Liste des colonnes avec leur type (String, Integer...) et leur caractère nullable.
Job	Programme ETL dessiné graphiquement ; Talend génère du code **Java** à l'exécution.
tFileInputDelimited	Composant d'**entrée** qui lit un fichier délimité (CSV, TXT).
tLogRow	Affiche les lignes dans la console (modes Simple, Table, Vertical).
tMap	Composant de **transformation** : jointures, filtres, expressions, sorties multiples.
Main / Lookup	Main = flux principal ; Lookup = table de référence utilisée pour la jointure (pointillés).
Inner Join / Left Outer Join	Inner : seulement les correspondances ; Left Outer : tout le flux principal.
Catch lookup inner join reject	Envoie vers une sortie les lignes rejetées par l'Inner Join.

Terme	Signification
tFileOutputDelimited / tFileOutputXML	Composants de **sortie** qui écrivent un fichier CSV / XML.
Code de sortie 0 / 1	0 = exécution sans erreur ; 1 = le job s'est arrêté sur une erreur.

4.3 Questions d'entraînement (avec réponses)

Q1. Que signifie ETL et quel est le rôle de chaque étape ?

Réponse : Extract (récupérer les données des sources), Transform (nettoyer et rendre cohérent), Load (charger dans l'entrepôt).

Q2. Différence entre extraction complète et incrémentale ?

Réponse : Complète : toutes les données à un instant donné (chargement initial). Incrémentale : uniquement les nouveautés/modifications depuis le dernier chargement (temps réel ou périodique).

Q3. Quels sont les 4 problèmes de qualité des données réelles ?

Réponse : Données incomplètes (manquantes), bruitées, incohérentes, dupliquées.

Q4. Citez 4 techniques pour les données manquantes.

Réponse : Ignorer l'enregistrement, remplir manuellement, remplacer par une constante (« unknown »), utiliser la moyenne ou la valeur la plus probable.

Q5. Différence entre conflit structurel et conflit de représentation ?

Réponse : Structurel : propriétés différentes pour le même objet (âge vs date de naissance). Représentation : même donnée, format différent (YYYY-MM-DD vs MM/DD/YY).

Q6. Différence entre Roll-up et Drill-down ?

Réponse : Roll-up agrège vers un niveau plus haut (jours → mois) ; Drill-down détaille vers un niveau plus bas (régions → villes).

Q7. Différence entre Slice et Dice ?

Réponse : Slice fixe **une** dimension à une valeur (tranche 2D) ; Dice filtre **plusieurs** dimensions (sous-cube).

Q8. Dans le Lab 5, pourquoi seulement 255 films en sortie de l'exercice 2 ?

Réponse : Parce que l'Inner Join ne garde que les films dont le directorID correspond à un réalisateur ; 1 200 films ont un directorID vide.

Q9. Comment récupérer les films rejetés dans l'exercice 3 ?

Réponse : Créer une 2e sortie dans le tMap avec « Catch lookup inner join reject = true ».

Q10. Pourquoi directorID et director_id doivent-ils avoir le même type ?

Réponse : Pour que la jointure du tMap compare des valeurs de même type (String = String), sinon erreur de compilation Java.

Q11. Quelle notion du cours illustre le .trim() ?

Réponse : Le traitement des données bruitées (erreurs de saisie : espaces superflus).